API SECURITY RISK ANALYSIS REPORT

(JSONPlaceholder API Assessment)

Prepared by:

Mohammed Fahad P

Cybersecurity Intern

Date: April 5, 2026

Classification: Internal / Educational Use Only

Assessment Type: Read-Only, Non-Destructive, Ethical

CONFIDENTIAL – For Authorized Personnel Only

1. Introduction

This report presents the findings of an API security assessment conducted on the publicly available JSONPlaceholder API (https://jsonplaceholder.typicode.com). JSONPlaceholder is a widely used fake REST API designed for testing and prototyping purposes. While it is not a production system, it exhibits several real-world security vulnerabilities that are commonly found in live API deployments. This assessment was carried out as part of a cybersecurity internship project to develop practical skills in API security analysis, vulnerability identification, and risk documentation.

All testing activities were performed in a strictly read-only and ethical manner. No data was altered, deleted, or submitted. No unauthorized systems were targeted. The purpose of this exercise is purely educational, and the findings are intended to illustrate how similar vulnerabilities in real-world APIs could pose significant security and privacy risks.

2. Objective

The primary objectives of this security assessment are as follows:

- To identify and document security vulnerabilities present in the target API.
- To assess the risk level associated with each identified vulnerability.
- To provide practical and actionable remediation recommendations.
- To demonstrate the application of API security assessment methodologies in a real-world scenario.
- To produce a professional security report suitable for a client-facing environment.

3. Methodology

The security assessment followed a structured, manual testing approach aligned with the OWASP API Security Top 10 framework. All tests were conducted using Postman, a widely adopted API testing platform. The assessment was limited to read-only HTTP GET requests and did not involve any form of active exploitation, fuzzing, injection, or automated scanning.

The methodology consisted of the following phases:

Phase 1: Reconnaissance & Endpoint Discovery

Publicly available API documentation was reviewed to identify the available endpoints, data structures, and access patterns. No credentials or special permissions were required to access the API, which itself was noted as an initial finding.

Phase 2: Manual Request Testing

Targeted GET requests were sent to the identified endpoints using Postman. Response headers, status codes, payload contents, and access patterns were carefully analyzed for signs of security misconfigurations.

Phase 3: Vulnerability Analysis

Each observed behavior was evaluated against known API security weaknesses, including those defined in the OWASP API Security Top 10. Findings were categorized by vulnerability type, assigned a risk level (High, Medium, or Low), and documented with supporting evidence.

Phase 4: Reporting

The findings, evidence, and remediation recommendations were compiled into this professional security report following industry-standard documentation practices.

4. API Tested

API Name: JSONPlaceholder
Base URL: https://jsonplaceholder.typicode.com
API Type: RESTful HTTP API (Public / Fake API for Testing)
Authentication Required: None
Testing Tool: Postman
Assessment Date: April 5, 2026

4.1 Endpoints Tested

Endpoint	Method	Description	Status	Response Size
/users	GET	Returns a list of all users with full profile data	200 OK	7.99 KB / 490ms
/users/1	GET	Returns detailed profile of a specific user by ID	200 OK	1.4 KB / 225ms
/posts	GET	Returns a list of all blog posts	200 OK	7.99 KB / 490ms

5. Findings & Risk Analysis

The assessment identified four distinct security vulnerabilities. A summary of all findings is presented in the table below, followed by detailed analysis of each finding.

ID	Finding	Risk Level	Category
F-01	No Authentication	HIGH	Authentication & Authorization
F-02	Excessive Data Exposure	MEDIUM	Information Disclosure
F-03	Lack of Rate Limiting	MEDIUM	API Security
F-04	Broken Access Control	MEDIUM	Access Control

5.1 Finding F-01: No Authentication

No Authentication or Authorization Mechanism [HIGH RISK]

Description

The API does not implement any form of authentication or authorization. All endpoints are publicly accessible without requiring an API key, bearer token, session cookie, or any other credential. Any anonymous user or automated system can send requests to the API and receive full data responses without restriction. This represents a fundamental violation of the principle of least privilege and exposes the API to unauthorized access.

Impact

In a real-world deployment, the absence of authentication would allow any malicious actor to freely access, harvest, and exfiltrate sensitive user data. Attackers could enumerate all user records, extract personal information, and use this data for social engineering, identity theft, phishing attacks, or targeted fraud. Without authentication, there is no way to trace or attribute requests to specific users, making forensic investigation after a breach extremely difficult.

Remediation

Implement robust API authentication using industry-standard mechanisms. OAuth 2.0 with Bearer Tokens (JWT) is recommended for modern REST APIs. All endpoints should require a valid access token before returning data. API keys with scoped permissions may be used for service-to-service communication. Access control policies should follow the principle of least privilege, granting users only the permissions necessary for their role.

5.2 Finding F-02: Excessive Data Exposure

Excessive Data Exposure via API Response [MEDIUM RISK]

Description

The API responses return significantly more data than is necessary for the intended use case. For example, a request to /users returns complete user profiles including full name, email address, phone number, home address (including zip code), geographic coordinates (latitude and longitude), company name, company tagline, and company business summary. No data filtering or field selection mechanism is provided to limit the response to only the data that is needed.

Impact

Excessive data exposure increases the attack surface of the API and can lead to data privacy violations. Even if a consumer application only displays a user's name and email, the full payload is transmitted over the network and available in the client's memory. This data can be

intercepted, logged by proxies, or extracted by malicious browser scripts. In a real system, exposing geographic coordinates and detailed addresses could enable physical harm to users.

Remediation

Implement server-side response filtering to return only the fields required for each specific use case. Adopt a data minimization strategy in line with privacy regulations such as GDPR and CCPA. Consider implementing API request parameters (e.g., ?fields=id,name,email) to allow clients to specify required fields. Sensitive fields such as address, phone, and geographic coordinates should be protected by additional access controls and never returned by default.

5.3 Finding F-03: Lack of Rate Limiting

No Rate Limiting on API Requests [MEDIUM RISK]

Description

The API does not enforce any rate limiting or request throttling. During testing, multiple rapid requests were sent to the /users and /posts endpoints in succession without any restriction, delay, or warning from the server. The API continued to respond with 200 OK status codes and full data payloads regardless of request frequency. No HTTP 429 (Too Many Requests) responses were observed at any point during the assessment.

Impact

The absence of rate limiting exposes the API to multiple attack vectors. Malicious actors could conduct automated scraping attacks to harvest all available user data in a short period. The API is also vulnerable to Denial of Service (DoS) attacks, in which a flood of requests could overwhelm the server and make it unavailable to legitimate users. Additionally, lack of rate limiting makes brute-force attacks against predictable resource identifiers (e.g., /users/1, /users/2, /users/3) highly feasible.

Remediation

Implement rate limiting at the API gateway or web server level. Define appropriate thresholds based on expected usage patterns (e.g., 100 requests per minute per IP or per API key). Return HTTP 429 (Too Many Requests) responses when limits are exceeded, and include a Retry-After header to inform clients when they may resume requests. Consider implementing exponential backoff policies and CAPTCHA challenges for high-frequency access patterns.

5.4 Finding F-04: Broken Access Control (Conceptual)

Broken Object-Level Access Control (BOLA/IDOR) [MEDIUM RISK]

Description

The API allows any user to access the profile data of any other user by simply modifying the user ID in the request URL (e.g., /users/1, /users/2, /users/3). No ownership validation or authorization check is performed to verify whether the requesting party has the right to access the specified resource. This vulnerability is classified as Broken Object-Level Authorization (BOLA), also known as Insecure Direct Object Reference (IDOR), and is listed as the most critical API vulnerability in the OWASP API Security Top 10.

Impact

In a production environment, this vulnerability would allow any authenticated (or unauthenticated, given Finding F-01) user to access the private data of any other user by

simply guessing or enumerating numeric user IDs. An attacker could systematically retrieve the profiles of all registered users without any restriction. This represents a severe privacy breach that could affect millions of users and result in significant regulatory fines under data protection laws.

Remediation

Implement server-side authorization checks on every object-level request. Verify that the authenticated user is the owner of the requested resource, or has been explicitly granted permission to access it, before returning any data. Avoid exposing predictable sequential integer IDs in API URLs; instead, use UUIDs (Universally Unique Identifiers) to reduce the effectiveness of enumeration attacks. Enforce object-level access control consistently across all API endpoints.

6. Evidence

The following screenshots were captured during the assessment using Postman. They provide direct evidence of the identified vulnerabilities. Each figure corresponds to a specific finding as referenced in Section 5.

Figure 1: /users Endpoint Response (Findings F-01, F-02, F-03, F-04)

The /users endpoint returns a full array of user objects without requiring any authentication token or API key. The response includes personally identifiable information (PII) such as email, phone, address, and geographic coordinates for each user.

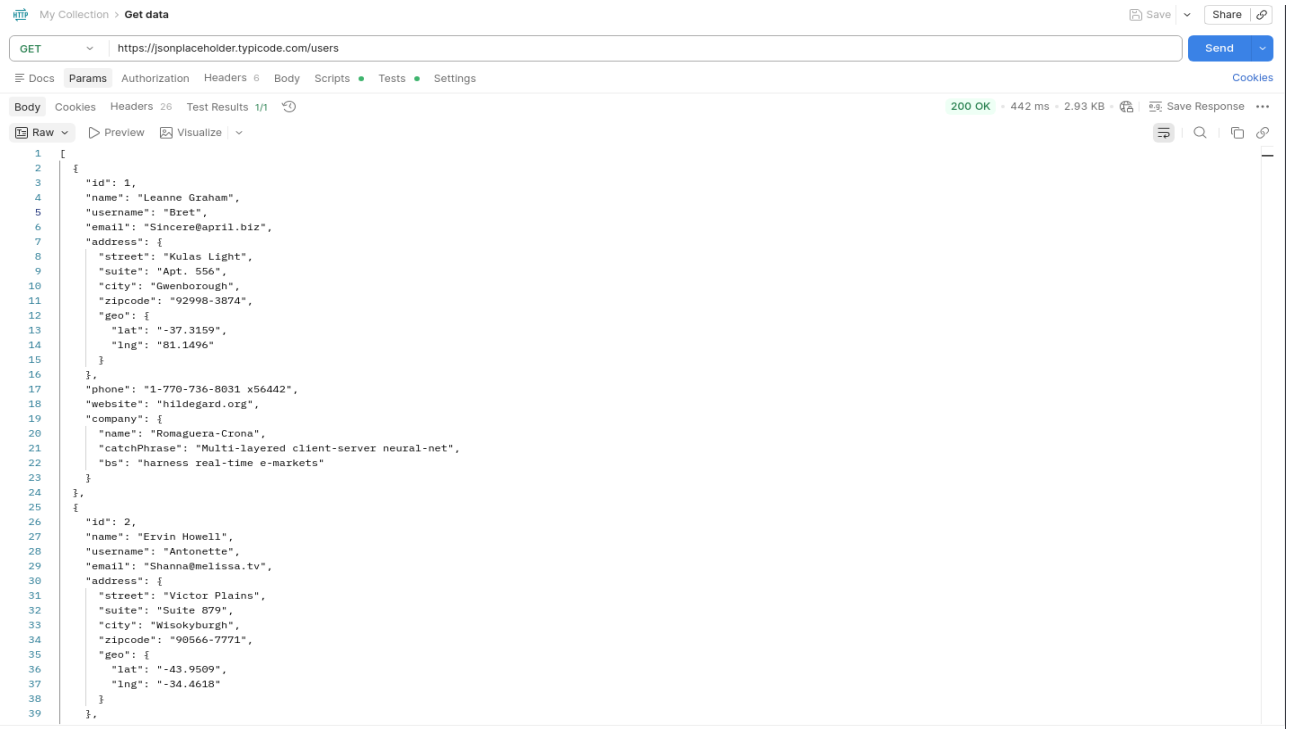


Figure 2: /users/1 Endpoint Response (Finding F-04 — BOLA/IDOR)

Accessing /users/1 returns the complete profile of a specific user. Changing the trailing integer to any other valid user ID (e.g., /users/2, /users/3) returns the profile of that user, with no authorization check performed. This demonstrates the Broken Object-Level Access Control vulnerability.

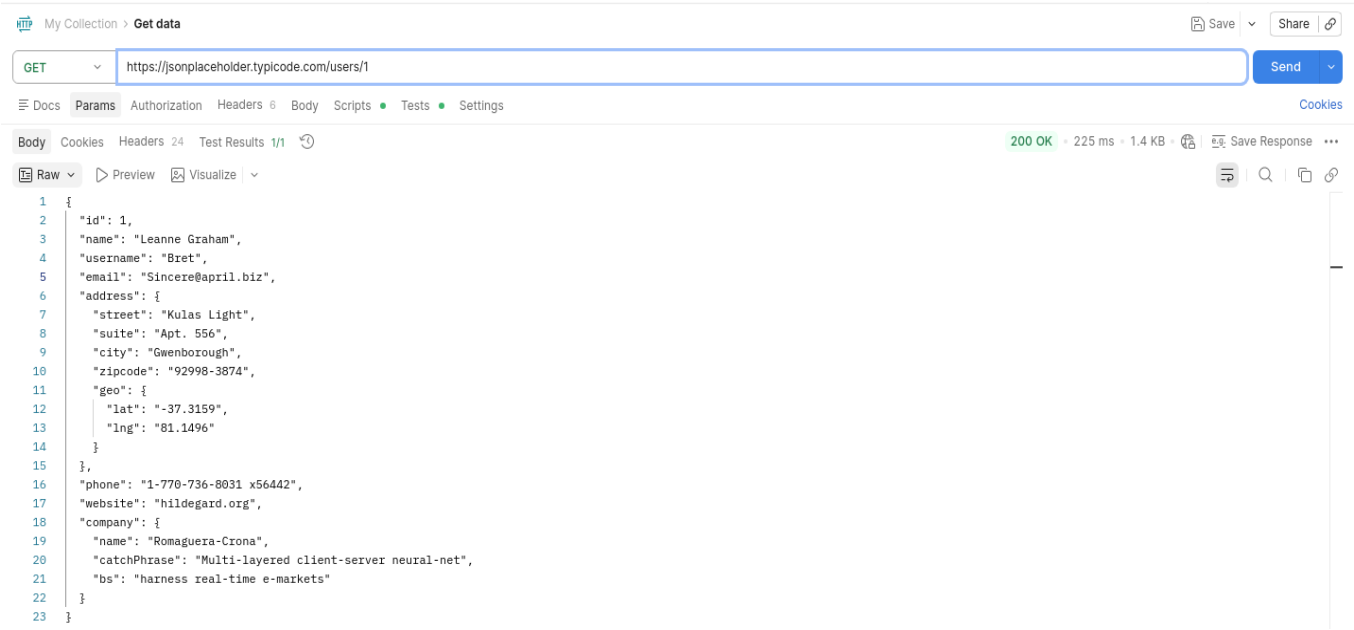
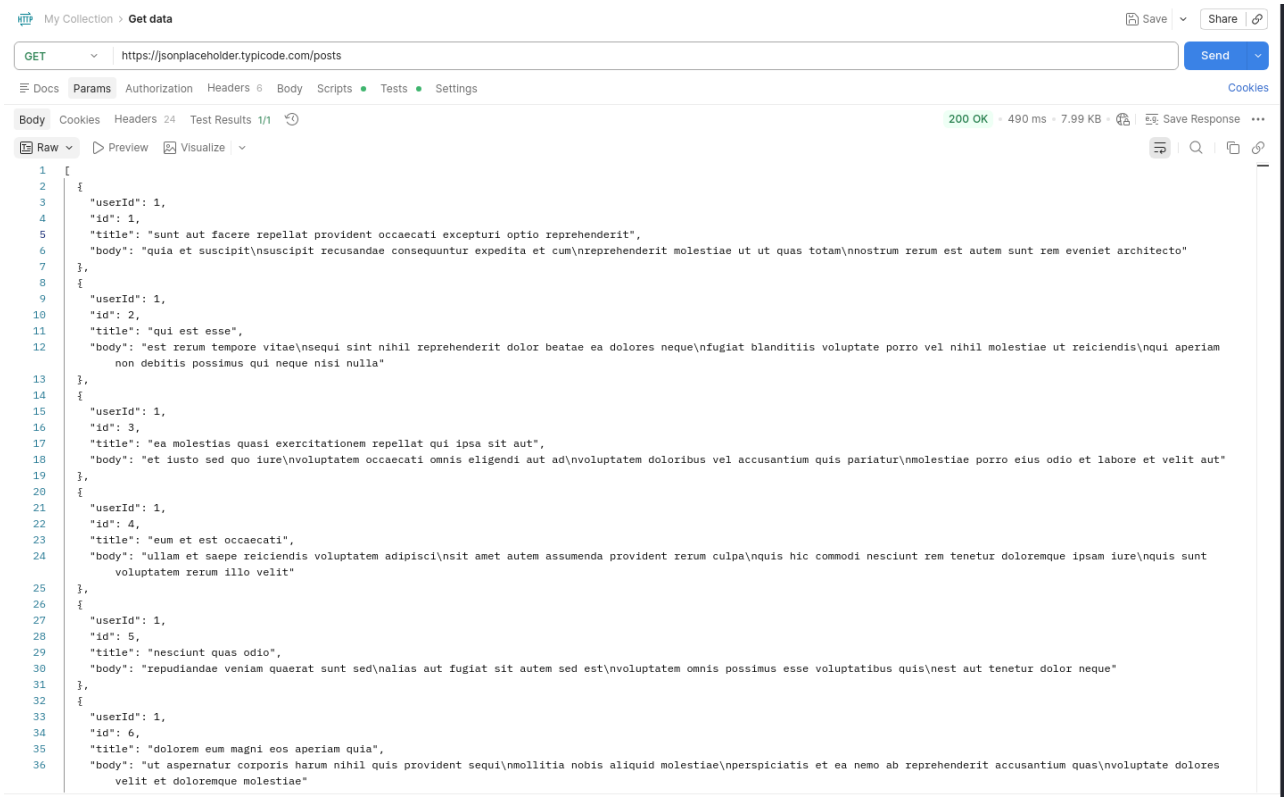


Figure 3: /posts Endpoint Response (Finding F-01)

The /posts endpoint returns all 100 blog post records in a single unauthenticated request. Each record includes a userId, post ID, title, and full body text. No token or credential is required to access this data.



7. Tools Used

Tool / Resource	Category	Purpose
Postman	API Testing Platform	Primary tool used for sending HTTP requests, inspecting responses, and capturing evidence
JSONPlaceholder	Target API	Fake REST API used as the target for this security assessment (https://jsonplaceholder.typicode.com)
OWASP API Security Top 10	Reference Framework	Industry-standard framework used to classify and evaluate identified vulnerabilities
Web Browser (DevTools)	Supplementary	Used to inspect HTTP response headers and verify lack of security headers

8. Conclusion

This security assessment of the JSONPlaceholder API identified four significant vulnerabilities spanning authentication, data privacy, access control, and request management. While JSONPlaceholder is designed as a testing tool and not a production system, the vulnerabilities documented in this report are representative of weaknesses commonly found in real-world APIs. The findings underscore the critical importance of implementing security controls from the earliest stages of API development.

The most critical finding (F-01: No Authentication) represents a foundational security gap that would, in a live environment, expose all system data to unauthorized parties. The supporting findings of Excessive Data Exposure (F-02), Lack of Rate Limiting (F-03), and Broken Object-Level Access Control (F-04) collectively demonstrate how the absence of standard security controls can compound risk across multiple attack vectors.

It is strongly recommended that any production API incorporating similar design patterns address these vulnerabilities prior to deployment. The remediation steps outlined in Section 5 of this report provide a practical roadmap for resolving each identified issue in accordance with industry best practices.

Ethical Statement

This assessment was conducted solely for educational purposes as part of a cybersecurity internship project. All tests were limited to read-only HTTP GET requests against a publicly available, non-production API. No data was modified, deleted, or exfiltrated. No unauthorized systems were accessed. The findings in this report are intended to demonstrate practical API security assessment skills and do not represent any malicious intent.

End of Report

Mohammed Fahad P | Cybersecurity Intern | April 5, 2026